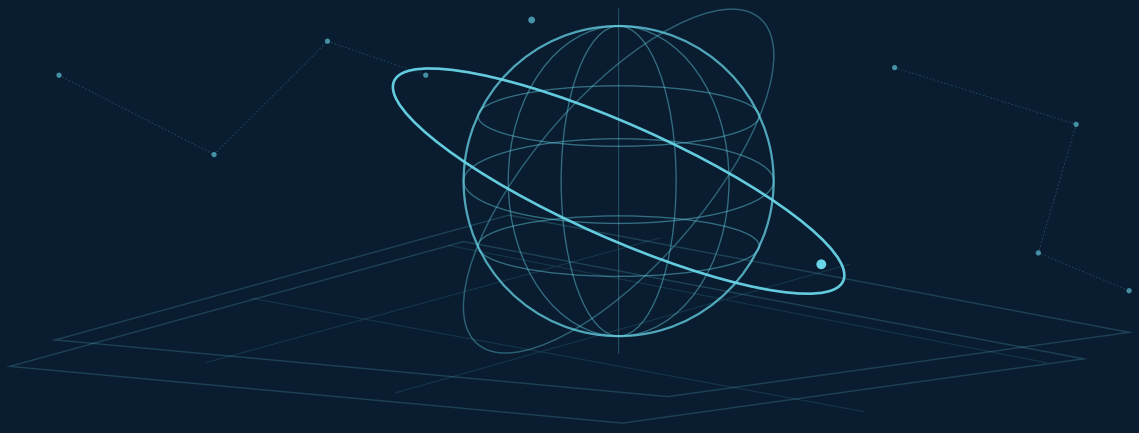


SPECIFICATION

Minimal request register

A complete small Atlas instance with one entity, validation, tool, case, requirement set, and web boundary.

ATLAS-EX-MINIMAL

Revision 2 / draft / Stable-v1.0.0

Document: ATLAS-EX-MINIMAL

Revision: 2

Status: draft

Language: en

Atlas version: Stable-v1.0.0

Profile version: Stable-v1.0.0

Translator: 1.0.0

Created: 2026-10-25

Updated: 2026-10-25

Authors

- Douglas de Oliveira Alkimim

Specification author

Keywords

- minimal
- request
- validation
- boundary

Context, actors, and scope

The request register retains requests identified by code. It is a declarative specification; no business application is built here.

The scope covers submitting one request. Retrieval, deletion, pricing, stock, authentication implementation, and multi-tenant operation are outside this example.

> The tool is deliberately abstract: it states a persistence contract without choosing storage technology.

This is a synthetic specification example. Source SRC-REQUEST-BRIEF is a teaching baseline, not evidence of stakeholder approval or a claimed Needs-to-L1 derivation.

ACTOR ACT-REQUESTER

Name

Requester

Kind: person

Description

A person who submits a request to the register.

TERM VOC-REQUEST-CODE

Term

Request code

Definition

The exact text identifier of a retained request.

Synonyms: none

Entities and validations

The entity holds request data and defines validity; it has no collaborators.

ENTITY ENT-REQUEST

Name

Request

Meaning

A request asks the register to retain a quantity of a catalog code. The request is identified by its code.

Identity

Description

Two requests are the same when their codes are equal.

Fields: FLD-REQUEST-CODE

Fields

FLD-REQUEST-CODE

Name

Request code

Type: text

Cardinality: 1

Required: true

Description

A non-empty code chosen by the requester.

Constraints

The code is preserved exactly; case is significant.

Examples

Valid

- REQ-001
- req-001

Invalid

-

FLD-REQUEST-QUANTITY

Name

Quantity

Type: integer

Cardinality: 1

Required: true

Description

The number of units requested.

Constraints

The value must be an integer greater than or equal to 1.

Examples

Valid

- 1
- 2

Invalid

- 0
- 1.5

Relationships: none

States: none

Transitions: none

Validations

VLD-REQUEST-QUANTITY

Name

Quantity is a positive integer

Scope: field

Statement

The quantity field is valid when its value is an integer greater than or equal to 1.

Inputs

Name

Quantity

Source: field

Meaning

The value supplied for FLD-REQUEST-QUANTITY.

Conditions

C-VLD-REQ-INT

Condition

The value is an integer.

Meaning

A fractional or textual value fails this condition.

C-VLD-REQ-MIN

Condition

The value is at least 1.

Meaning

Zero and negative values fail this condition.

Results

Outcome: pass

Description

Both conditions hold; the field accepts the value.

Outcome: fail

Description

At least one condition fails; the field rejects the value.

Flow: GRAPH-VLD-REQUEST-QUANTITY

VLD-REQUEST-CODE

Name

Request code is non-empty text

Scope: field

Statement

The code must be text containing at least one character. Preserve the code exactly; case is significant.

Inputs

Name

Code

Source: field

Meaning

FLD-REQUEST-CODE candidate.

Conditions

C-VLD-REQ-CODE-NONEMPTY

Condition

The code is text with at least one character.

Meaning

An empty or non-text code fails.

Results

Outcome: pass

Description

Accept unchanged.

Outcome: fail

Description

Refuse before persistence.

Flow: GRAPH-VLD-REQUEST-CODE

Examples

EX-REQUEST-VALID

Kind: valid

Description

A request with a non-empty code and a positive quantity.

Data

```
{
"code": "REQ-001",
"quantity": 2
}
```

EX-REQUEST-INVALID

Kind: invalid

Description

A request whose quantity is zero.

Data

```
{
"code": "REQ-001",
"quantity": 0
}
```

Assumptions: ASM-REQUEST-STORE-UNIQUE

Open issues: OI-REQUEST-LATENCY

Abstract tools

The tool exposes a capability and a contract, not an implementation.

TOOL TOL-REQUEST-STORE

Name

Request store

Purpose

Retain request records durably. The contract does not choose a database, transport, or file layout.

Operations

OP-REQUEST-STORE

Name

Store request

Purpose

Create one retained request record when its code is unused.

Inputs

Name

Request code

Type: text

Meaning

The code to retain.

Required: true

Name

Quantity

Type: integer

Meaning

An already validated positive integer.

Required: true

Conditions

The code is not allocated to a retained request.

The store is available.

Inputs described as ready entities already satisfy canonical entity invariants. Do not duplicate those validations inside this tool. When constructing entities from raw retained records, use the canonical entity hydration rules; operation-specific external conditions remain this tool's responsibility.

Results

Name

Stored request

Type: REQUEST

Meaning

The retained request with its code and quantity.

Guarantees

After a successful result, the record is readable by code in the same retained store.

Effects

One request record is added to the retained store.

Failures

Code: DUPLICATE_CODE

Meaning

The code is already allocated.

Consumer distinction

The consumer must distinguish this from a successful store and from an unavailable store.

Code: STORE_UNAVAILABLE

Meaning

The store did not confirm the write.

Consumer distinction

Return write_outcome=not_applied only when the tool can guarantee no write; otherwise return write_outcome=unknown. Neither outcome is confirmation. An unknown outcome must not trigger an automatic retry.

Guarantees

A successful result confirms one retained record.

A duplicate-code failure leaves the existing record unchanged.

Use cases

Functional and non-functional requirements are declared in the case.

USE CASE `UC-REQUEST-SUBMIT`

Name

Submit request

Objective

Retain one valid request and confirm the stored record.

Actors

Actor: ACT-REQUESTER

Permission

Permitted to submit requests.

Description

The requester supplies a code and quantity.

Resolve this permission from trusted context for the concrete initiating caller; deny absent or unresolved evidence.

Trigger

The requester submits the request form.

Context

One request is submitted against the single request register.

Caller context must contain a resolved authenticated principal, the relevant case permissions, and a validity status. Its origin is the application's trusted identity-and-permission authority; request fields and caller-supplied role strings are not trusted authorization evidence. Anonymous execution is not permitted. Missing, unresolved, expired or insufficient context is denied before any validation, tool access or effect.

The initiating actor is ACT-REQUESTER. The actor is a role; the caller is the concrete principal. The identity/authentication mechanism and channel encoding remain later design decisions.

Inputs

Name

Request code

Type: text

Meaning

The code that identifies the request.

Required: true

Name

Quantity

Type: integer

Meaning

The number of units requested.

Required: true

Entry conditions

Requests may arrive without valid caller context; the explicit denial path handles them before any collaborator is invoked.

Entities

Entity: ENT-REQUEST

Validations: VLD-REQUEST-QUANTITY, VLD-REQUEST-CODE

Purpose

Hold the supplied code and quantity and validate the quantity.

Tools

Tool: TOL-REQUEST-STORE

Operation: OP-REQUEST-STORE

Purpose

Retain the request after validation succeeds.

Calls: none

Outcomes

`OUT-REQUEST-CONFIRMED`

Kind: success

Description

The request is retained and confirmed.

Effects

One new request record exists in the store.

`OUT-REQUEST-INVALID`

Kind: refusal

Description

The code or quantity is invalid; the request is refused.

Effects

No store call and no retained record.

OUT-REQUEST-DENIED

Kind: refusal

Description

The actor is absent or not permitted.

Effects

No validation, store call, or retained record.

OUT-REQUEST-STORE-FAILURE

Kind: failure

Description

The store reports that it is unavailable.

Effects

No confirmed request is reported.

OUT-REQUEST-DUPLICATE

Kind: refusal

Description

The request code is already retained.

Effects

This invocation leaves the existing request unchanged.

Functional requirements

RF-REQUEST-01

Statement

The case admits only an actor with the requester permission and refuses every other actor before any validation, store call, or effect.

Scope: case

Applies to: UC-REQUEST-SUBMIT, INT-REQUEST-SUBMIT

Flow references: N-REQ-PERM, N-REQ-DENY, E-REQ-2

RF-REQUEST-02

Statement

The case applies VLD-REQUEST-QUANTITY to the supplied quantity and, when the validation reports fail, refuses the request with the invalid-data outcome and no store call.

Scope: case

Applies to: UC-REQUEST-SUBMIT, INT-REQUEST-SUBMIT

Flow references: N-REQ-VALIDATE, N-REQ-QTYDEC, N-REQ-INVALID, E-REQ-4, E-REQ-5

RF-REQUEST-03

Statement

After a passing validation, the case calls OP-REQUEST-STORE with the code and quantity; a stored result produces the confirmation outcome.

Scope: operation

Applies to: UC-REQUEST-SUBMIT, OP-REQUEST-STORE

Flow references: N-REQ-STORE, E-REQ-6, E-REQ-7

RF-REQUEST-04

Statement

When OP-REQUEST-STORE reports STORE_UNAVAILABLE, the case reports the store-unavailable failure and does not claim a confirmed request.

Preserve whether the tool guarantees not_applied or reports unknown. A lost confirmation is not evidence of no write; do not automatically retry.

Scope: operation

Applies to: UC-REQUEST-SUBMIT, OP-REQUEST-STORE

Flow references: N-REQ-STOREERROR, E-REQ-8

RF-REQUEST-05

Statement

A confirmed request result contains the supplied code and quantity and identifies the new record.

Scope: interaction

Applies to: UC-REQUEST-SUBMIT, INT-REQUEST-SUBMIT

Flow references: N-REQ-SUCCESS, E-REQ-7

RF-REQUEST-06

Statement

When OP-REQUEST-STORE reports DUPLICATE_CODE, refuse the duplicate and leave the existing request unchanged; do not report confirmation.

Scope: operation

Applies to: UC-REQUEST-SUBMIT, OP-REQUEST-STORE

Flow references: N-REQ-DUPLICATE, E-REQ-DUPLICATE

RF-REQUEST-07

Statement

After authorization and before quantity validation, apply VLD-REQUEST-CODE. Refuse invalid code without storage; preserve valid code exactly.

Scope: case

Applies to: UC-REQUEST-SUBMIT

Flow references: N-REQ-INVALID, N-REQ-CODE, N-REQ-CODE-VALID, E-REQ-CODE-2, E-REQ-CODE-3

Nonfunctional requirements

RNF-REQUEST-01

Scope: interaction

Applies to: INT-REQUEST-SUBMIT

Statement

For inputs within the declared field limits and an available store, the boundary presents a confirmed or refused result within 2 seconds of the submit stimulus.

Verifiable criterion

The elapsed time from the submit stimulus to the visible result is at most 2 seconds.

Verification

Run 30 sequential submissions with valid and invalid quantities against the declared local configuration and record the result time for each.

Metric

Name

Visible result time

Unit

seconds

Target: 2

Operator: <=

Measurement

Start

The submit stimulus is accepted by the boundary.

End

The result state is visible to the actor.

References: SRC-REQUEST-BRIEF

Applicability

Origin: SRC-REQUEST-BRIEF. This is a synthetic teaching obligation; any numeric target and scenario count are illustrative assumptions, not an accepted production SLA or a measurement made by this repository.

Flows

Main: GRAPH-REQUEST-SUBMIT

Subflows: none

Flow text

Start: the case receives the request and actor. If the actor is not permitted, refuse with no effects (RF-REQUEST-01).

Apply VLD-REQUEST-CODE before the quantity check. An invalid code is refused without storage (RF-REQUEST-07).

Apply VLD-REQUEST-QUANTITY (RF-REQUEST-02). A fail result refuses the request as invalid data.

On pass, call OP-REQUEST-STORE (RF-REQUEST-03). A stored result produces the confirmed outcome (RF-REQUEST-05). STORE_UNAVAILABLE produces the store-unavailable failure (RF-REQUEST-04).

DUPLICATE_CODE is the duplicate refusal with no change to the preexisting request (RF-REQUEST-06). A write failure retains its known-not-applied versus unknown distinction (RF-REQUEST-04).

Acceptance criteria

AC-REQUEST-01

Statement

A requester with a valid code and quantity of 2 receives a confirmed result; a later lookup of the code returns quantity 2.

Verifies: RF-REQUEST-02, RF-REQUEST-03, RF-REQUEST-05

AC-REQUEST-02

Statement

A quantity of 0 is refused as invalid data, and no record is created.

Verifies: RF-REQUEST-02

AC-REQUEST-03

Statement

An actor absent from the permitted set is refused before validation or store access.

Verifies: RF-REQUEST-01

AC-REQUEST-04

Statement

Submit the same code twice. The second attempt reports duplicate-code refusal and preserves the first retained quantity.

Verifies: RF-REQUEST-06

AC-REQUEST-05

Statement

If a store write has an unknown final outcome, return the failure without claiming success or no write and without automatic retry.

Verifies: RF-REQUEST-04

AC-REQUEST-06

Statement

An empty code is refused before quantity validation and store invocation.

Verifies: RF-REQUEST-07

AC-REQUEST-SUBMIT-QUALITY

Statement

In the declared didactic configuration, demonstrate the canonical quality criterion and its stated verification procedure; preserve its status as a teaching assumption, not a measured production claim.

Verifies: RNF-REQUEST-01

Traceability

TRC-REQUEST-01

Source: RF-REQUEST-02

Targets: VLD-REQUEST-QUANTITY, AC-REQUEST-02

Note

The case applies the entity validation and states the refusal.

TRC-REQUEST-02

Source: RF-REQUEST-03

Targets: OP-REQUEST-STORE, AC-REQUEST-01

Note

The abstract tool contract carries the persistence guarantee.

TRC-REQUEST-SUBMIT-ORIGIN

Source: SRC-REQUEST-BRIEF

Targets: RF-REQUEST-01, RF-REQUEST-02, RF-REQUEST-03, RF-REQUEST-04, RF-REQUEST-05, RF-REQUEST-06, RF-REQUEST-07, RNF-REQUEST-01

Note

Versioned synthetic origin for these example obligations.

Assumptions: ASM-REQUEST-STORE-UNIQUE

Sources: SRC-REQUEST-BRIEF

Open issues: OI-REQUEST-LATENCY

Interaction boundaries

The boundary maps the case inputs and results to a web form.

BOUNDARY BND-REQUEST-WEB

Name

Request register web form

Modality: web

Purpose

Expose the submit-request case through one form.

Origin classification: this channel is an illustrative solution proposal. Its form, command syntax, layout and encoding are outside the L1 conformance claim. The referenced cases define domain inputs and outcomes independently of this channel.

Audience

Requesters using the local register.

Interactions

INT-REQUEST-SUBMIT

Name

Submit request form

Description

A form with Request code, Quantity, and Submit request. The result area presents confirmation, invalid data, duplicate code, or store failure.

Illustration only; no channel-specific UI or command choice is promoted to a domain RF by appearing here.

Use case: UC-REQUEST-SUBMIT

Trigger

The requester activates Submit request.

Input mapping

Source: form.code

Target: UC-REQUEST-SUBMIT.input.code

Meaning

Maps the Request code field to the case code input.

Source: form.quantity

Target: UC-REQUEST-SUBMIT.input.quantity

Meaning

Maps the Quantity field to the case quantity input.

Result mapping

Source: OUT-REQUEST-CONFIRMED

Target: result.confirmation

Meaning

The request is retained and confirmed.

Source: OUT-REQUEST-INVALID

Target: result.invalid_data

Meaning

The code or quantity is invalid; the request is refused.

Source: OUT-REQUEST-STORE-FAILURE

Target: result.store_unavailable

Meaning

The store reports that it is unavailable.

Source: OUT-REQUEST-DENIED

Target: result.out-request-denied

Meaning

The actor is absent or not permitted.

Source: OUT-REQUEST-DUPLICATE

Target: result.out-request-duplicate

Meaning

The request code is already retained.

Requirement refs: RF-REQUEST-01, RF-REQUEST-02, RF-REQUEST-03, RF-REQUEST-04, RF-REQUEST-05, RF-REQUEST-06, RF-REQUEST-07

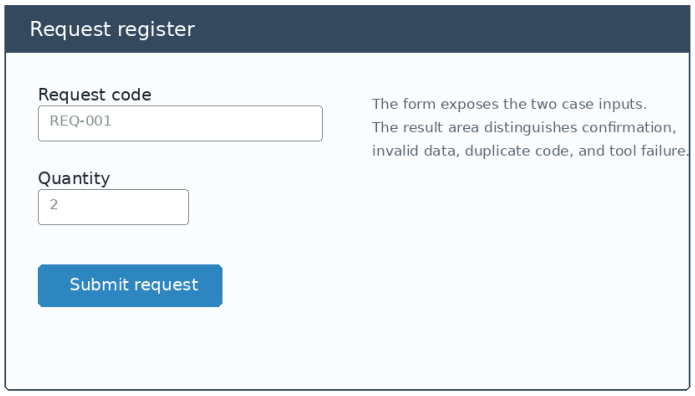
Resources: RES-REQUEST-WIRE, RES-REQUEST-FLOW

Examples

Wireframe with Request code, Quantity, and Submit request.

Alternative text:

A form with a Request code field, a Quantity field, and a Submit request button.



Assumptions: ASM-REQUEST-STORE-UNIQUE

Sources: SRC-REQUEST-BRIEF

Open issues: OI-REQUEST-LATENCY

Appendix: graphs

Graph `GRAPH-VLD-REQUEST-QUANTITY`

Quantity validation

Kind: flow

Nodes

Node `N-VLD-REQ-START` (start)

Start

Node `N-VLD-REQ-READ` (action)

Read quantity

Node `N-VLD-REQ-TYPE` (decision)

Integer?

Node `N-VLD-REQ-MIN` (decision)

At least 1?

Node `N-VLD-REQ-PASS` (result)

Valid

Node `N-VLD-REQ-FAIL-TYPE` (refusal)

Invalid: not an integer

Node `N-VLD-REQ-FAIL-MIN` (refusal)

Invalid: less than 1

Edges

Edge E-VLD-REQ-1: N-VLD-REQ-START → N-VLD-REQ-READ

Requirement refs: none

Edge E-VLD-REQ-2: N-VLD-REQ-READ → N-VLD-REQ-TYPE

Requirement refs: none

Edge E-VLD-REQ-3: N-VLD-REQ-TYPE → N-VLD-REQ-FAIL-TYPE

Label

no

Requirement refs: none

Edge E-VLD-REQ-4: N-VLD-REQ-TYPE → N-VLD-REQ-MIN

Label

yes

Requirement refs: none

Edge E-VLD-REQ-5: N-VLD-REQ-MIN → N-VLD-REQ-FAIL-MIN

Label

no

Requirement refs: none

Edge E-VLD-REQ-6: N-VLD-REQ-MIN → N-VLD-REQ-PASS

Label

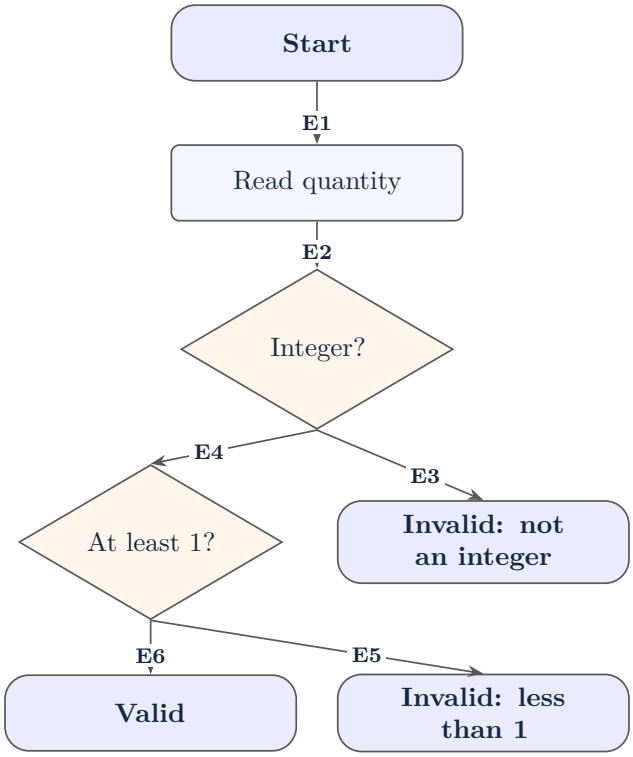
yes

Requirement refs: none
Topology of GRAPH-VLD-REQUEST-QUANTITY:
Nodes:
- N-VLD-REQ-START (start)
- N-VLD-REQ-READ (action)
- N-VLD-REQ-TYPE (decision)
- N-VLD-REQ-MIN (decision)
- N-VLD-REQ-PASS (result)
- N-VLD-REQ-FAIL-TYPE (refusal)
- N-VLD-REQ-FAIL-MIN (refusal)

Edges:
- E-VLD-REQ-1: N-VLD-REQ-START -> N-VLD-REQ-READ
- E-VLD-REQ-2: N-VLD-REQ-READ -> N-VLD-REQ-TYPE
- E-VLD-REQ-3: N-VLD-REQ-TYPE -> N-VLD-REQ-FAIL-TYPE
- E-VLD-REQ-4: N-VLD-REQ-TYPE -> N-VLD-REQ-MIN
- E-VLD-REQ-5: N-VLD-REQ-MIN -> N-VLD-REQ-FAIL-MIN
- E-VLD-REQ-6: N-VLD-REQ-MIN -> N-VLD-REQ-PASS

Quantity validation

GRAPH-VLD-REQUEST-QUANTITY



Edge key

E1 E-VLD-REQ-1
E2 E-VLD-REQ-2
E3 E-VLD-REQ-3 — no
E4 E-VLD-REQ-4 — yes
E5 E-VLD-REQ-5 — no
E6 E-VLD-REQ-6 — yes

Graph GRAPH-REQUEST-SUBMIT

Submit request

Kind: flow

Nodes

Node N-REQ-START (start)

Start

Node N-REQ-PERM (decision)

Actor permitted?

Requirement refs: RF-REQUEST-01

Node N-REQ-DENY (refusal)

Refuse: actor not permitted

Requirement refs: RF-REQUEST-01

Node N-REQ-VALIDATE (validation)

Apply VLD-REQUEST-QUANTITY

Call: validation VLD-REQUEST-QUANTITY

Requirement refs: RF-REQUEST-02

Node N-REQ-QTYDEC (decision)

Quantity valid?

Requirement refs: RF-REQUEST-02

Node N-REQ-INVALID (refusal)

Invalid code or quantity

Requirement refs: RF-REQUEST-02, RF-REQUEST-07

Node N-REQ-STORE (operation)

Store request

Call: operation OP-REQUEST-STORE

Requirement refs: RF-REQUEST-03

Node N-REQ-STOREERROR (failure)

Report STORE_UNAVAILABLE

Requirement refs: RF-REQUEST-04

Node N-REQ-SUCCESS (result)

Request confirmed

Requirement refs: RF-REQUEST-05

Node N-REQ-DUPLICATE (refusal)

Duplicate code; no change

Requirement refs: RF-REQUEST-06

Node N-REQ-CODE (validation)

Apply code validity

Call: validation VLD-REQUEST-CODE

Requirement refs: RF-REQUEST-07

Node N-REQ-CODE-VALID (decision)

Code valid?

Requirement refs: RF-REQUEST-07

Edges

Edge E-REQ-1: N-REQ-START → N-REQ-PERM

Requirement refs: none

Edge E-REQ-2: N-REQ-PERM → N-REQ-DENY

Label

no

Requirement refs: RF-REQUEST-01

Edge E-REQ-3: N-REQ-PERM → N-REQ-CODE

Label

yes

Requirement refs: none

Edge E-REQ-4: N-REQ-VALIDATE → N-REQ-QTYDEC

Requirement refs: RF-REQUEST-02

Edge E-REQ-5: N-REQ-QTYDEC → N-REQ-INVALID

Label

fail

Requirement refs: RF-REQUEST-02

Edge E-REQ-6: N-REQ-QTYDEC → N-REQ-STORE

Label

pass

Requirement refs: RF-REQUEST-03

Edge E-REQ-7: N-REQ-STORE → N-REQ-SUCCESS

Label

stored

Requirement refs: RF-REQUEST-03, RF-REQUEST-05

Edge E-REQ-8: N-REQ-STORE → N-REQ-STOREERROR

Label

STORE_UNAVAILABLE

Requirement refs: RF-REQUEST-04

Edge E-REQ-DUPLICATE: N-REQ-STORE → N-REQ-DUPLICATE

Label

DUPLICATE_CODE

Requirement refs: RF-REQUEST-06

Edge E-REQ-CODE-1: N-REQ-CODE → N-REQ-CODE-VALID

Requirement refs: none

Edge E-REQ-CODE-2: N-REQ-CODE-VALID → N-REQ-VALIDATE

Label

pass

Requirement refs: RF-REQUEST-07

Edge E-REQ-CODE-3: N-REQ-CODE-VALID → N-REQ-INVALID

Label

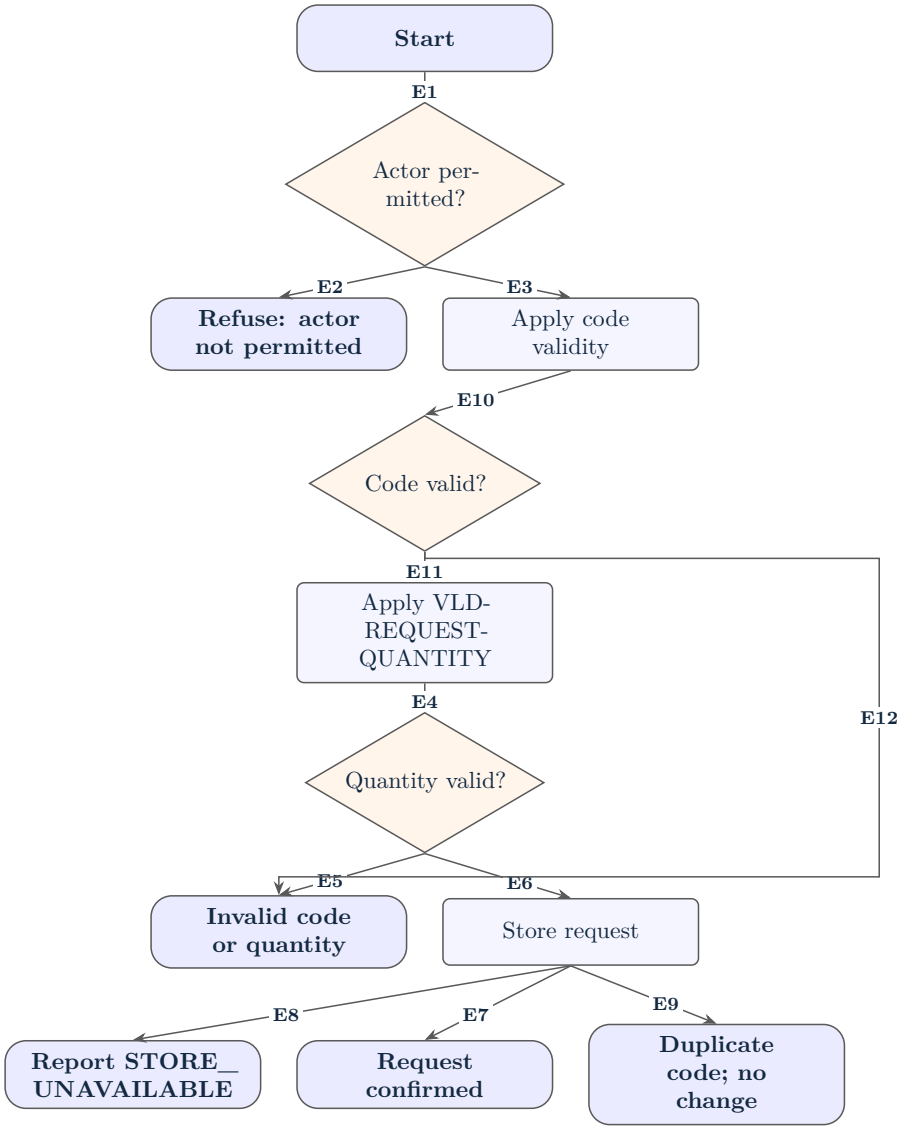
fail

Requirement refs: RF-REQUEST-07
Topology of GRAPH-REQUEST-SUBMIT:
Nodes:
- N-REQ-START (start)
- N-REQ-PERM (decision)
- N-REQ-DENY (refusal)
- N-REQ-VALIDATE (validation)
- N-REQ-QTYDEC (decision)
- N-REQ-INVALID (refusal)
- N-REQ-STORE (operation)
- N-REQ-STOREERROR (failure)
- N-REQ-SUCCESS (result)
- N-REQ-DUPLICATE (refusal)
- N-REQ-CODE (validation)
- N-REQ-CODE-VALID (decision)

Edges:
- E-REQ-1: N-REQ-START -> N-REQ-PERM
- E-REQ-2: N-REQ-PERM -> N-REQ-DENY
- E-REQ-3: N-REQ-PERM -> N-REQ-CODE
- E-REQ-4: N-REQ-VALIDATE -> N-REQ-QTYDEC
- E-REQ-5: N-REQ-QTYDEC -> N-REQ-INVALID
- E-REQ-6: N-REQ-QTYDEC -> N-REQ-STORE
- E-REQ-7: N-REQ-STORE -> N-REQ-SUCCESS
- E-REQ-8: N-REQ-STORE -> N-REQ-STOREERROR
- E-REQ-DUPLICATE: N-REQ-STORE -> N-REQ-DUPLICATE
- E-REQ-CODE-1: N-REQ-CODE -> N-REQ-CODE-VALID
- E-REQ-CODE-2: N-REQ-CODE-VALID -> N-REQ-VALIDATE
- E-REQ-CODE-3: N-REQ-CODE-VALID -> N-REQ-INVALID

Submit request

GRAPH-REQUEST-SUBMIT



Edge key

E1 E-REQ-1
E2 E-REQ-2 — no
E3 E-REQ-3 — yes
E4 E-REQ-4
E5 E-REQ-5 — fail
E6 E-REQ-6 — pass
E7 E-REQ-7 — stored
E8 E-REQ-8 — STORE_UNAVAILABLE
E9 E-REQ-DUPLICATE — DUPLICATE_CODE
E10 E-REQ-CODE-1
E11 E-REQ-CODE-2 — pass
E12 E-REQ-CODE-3 — fail

Graph GRAPH-VLD-REQUEST-CODE

Request code validity

Kind: flow

Nodes

Node N-VLD-REQ-CODE-START (start)

Receive code

Node N-VLD-REQ-CODE-CHECK (decision)

Non-empty text?

Node N-VLD-REQ-CODE-PASS (result)

Pass unchanged

Node N-VLD-REQ-CODE-FAIL (refusal)

Invalid code

Edges

Edge E-VLD-REQ-CODE-1: N-VLD-REQ-CODE-START → N-VLD-REQ-CODE-CHECK

Requirement refs: none

Edge E-VLD-REQ-CODE-2: N-VLD-REQ-CODE-CHECK → N-VLD-REQ-CODE-PASS

Label

yes

Requirement refs: none

Edge E-VLD-REQ-CODE-3: N-VLD-REQ-CODE-CHECK → N-VLD-REQ-CODE-FAIL

Label

no

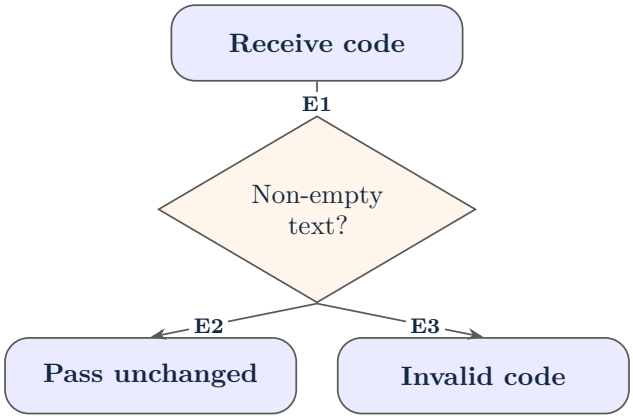
Requirement refs: none

Topology of GRAPH-VLD-REQUEST-CODE:
Nodes:
- N-VLD-REQ-CODE-START (start)
- N-VLD-REQ-CODE-CHECK (decision)
- N-VLD-REQ-CODE-PASS (result)
- N-VLD-REQ-CODE-FAIL (refusal)

Edges:
- E-VLD-REQ-CODE-1: N-VLD-REQ-CODE-START -> N-VLD-REQ-CODE-CHECK
- E-VLD-REQ-CODE-2: N-VLD-REQ-CODE-CHECK -> N-VLD-REQ-CODE-PASS
- E-VLD-REQ-CODE-3: N-VLD-REQ-CODE-CHECK -> N-VLD-REQ-CODE-FAIL

Request code validity

GRAPH-VLD-REQUEST-CODE



Edge key

E1 E-VLD-REQ-CODE-1
E2 E-VLD-REQ-CODE-2 — yes
E3 E-VLD-REQ-CODE-3 — no

Appendix: resources

Resource RES-REQUEST-WIRE

Path: assets/request-form.svg

Media type: image/svg+xml

SHA-256: f5a97fb7cb43af9681b2083fe1deebe38bb3eda24adb947c3314317fc8a24de9

Drawing source

Path: assets/request-form.svg

Revision: 1

Tool: scripts/build_examples.py

Caption

Request form wireframe: Request code, Quantity, and Submit request.

Alternative text

A form with a Request code field, a Quantity field, and a Submit request button. A result area reports confirmation, invalid data, duplicate code, or tool failure.

Text present in the image

- Request code
- Quantity
- Submit request

Variants

Path: assets/request-form.png

Media type: image/png

Sha256: 5dd57bf8bccc9bcd9b68e1be1ea50163777caf93d83f6fbfec1a099eb1d84347

Size: 22242

Relation: publication

Derived from: assets/request-form.svg

Producer: scripts/build_examples.py

Resource RES-REQUEST-FLOW

Path: assets/request-flow.svg

Media type: image/svg+xml

SHA-256: 71f3a97b667a88041cc5812da8c8e7e1ddaced2f9c835e56c2c7d6d1b4252206

Drawing source

Path: assets/request-flow.svg

Revision: 1286331986e92aa70d90981f34da7aaa1dc45b09fa69106b4717745a2a095c0a

Tool: scripts/build_examples.py

Derived from graph: GRAPH-REQUEST-SUBMIT

Caption

Submission flow with authorization, code and quantity validation, persistence, duplicate refusal and explicit write failure. The edge key identifies every structured edge.

Alternative text

Every node and edge of GRAPH-REQUEST-SUBMIT, with wrapped labels and an edge key. The text and graph records provide the complete accessible description.

Text present in the image

- Start
- Actor permitted?
- Refuse: actor not permitted
- Apply VLD-REQUEST-QUANTITY
- Quantity valid?
- Invalid code or quantity
- Store request
- Report STORE_UNAVAILABLE
- Request confirmed
- Duplicate code; no change
- Apply code validity
- Code valid?
- no
- yes
- fail
- pass
- stored
- STORE_UNAVAILABLE
- DUPLICATE_CODE
- pass
- fail

Variants

Path: assets/request-flow.png

Media type: image/png

Sha256: 53545111afdda2bcf5b113667ef14dc2233ca3f7e5e730749d3f50473ed81f8b

Size: 73547

Relation: publication

Derived from: assets/request-flow.svg

Producer: scripts/build_examples.py

Appendix: assumptions

ASM-REQUEST-STORE-UNIQUE

Statement

A future implementation of the request store can enforce code uniqueness atomically.

Status: open

Affected: TOL-REQUEST-STORE, UC-REQUEST-SUBMIT

Appendix: decisions

DEC-MIN-TOOL-ABSTRACTION

Title

Keep the store abstract

Context

The example needs persistence but must not choose a technology.

Decision

Define OP-REQUEST-STORE as a tool contract only.

Alternatives

Specify SQLite tables and SQL statements.

Consequences

Implementations may choose storage, but must satisfy the contract.

Status: accepted

Refs: TOL-REQUEST-STORE

Appendix: open issues

OI-REQUEST-LATENCY

Title

Confirm the latency target

Description

The 2-second target is a verifiable example target; a production adoption must confirm it with stakeholders.

Status: open

Refs: RNF-REQUEST-01

Appendix: sources

SRC-REQUEST-BRIEF

Title

Illustrative request-register brief

Uri: null

Revision: 2

Accessed: null

Note

Internal teaching source; not a commissioned requirement.

The authorization policy, interaction sketches, and quality targets are didactic choices, not observed business facts.

Appendix: history

HIST-REQUEST-01

Version: 1

Date: 2026-10-25

Description

Initial reference example for the first Atlas Specifications edition, Alpha-v0.1.0. This synthetic example demonstrates the model and its representations; it does not assert stakeholder approval or execution of the described business application.

HIST-MIGRATION-STABLE

Version: 2

Date: 2026-10-25

Description

Explicit editorial migration from Atlas Alpha-v0.1.0 revision 1 to Atlas Stable-v1.0.0 revision 2. Prepared by Codex under Douglas de Oliveira Alkimim's execution request. Model/profile/envelope and producer identities change; existing concept IDs, domain contracts, authored history and resources are retained. This is not format conversion, a Needs-to-L1 derivation, or new stakeholder approval.

Derived summary: 5 sections; 1 actors; 1 entities; 1 tools; 1 use cases; 1 boundaries; 3 graphs; 2 resources.